

Seminario sul paper “MEV Mitigation Approaches in Ethereum and Layer-2 Chains”

Buongiorno a tutti.

In questo seminario parlerò del paper “*MEV Mitigation Approaches in Ethereum and Layer-2 Chains: A Comprehensive Survey*”, pubblicato nel 2024 su IEEE Access.

Il MEV si verifica quando miner o validatori manipolano l'ordine delle transazioni all'interno di un blocco per ottenere un profitto personale. Un esempio tipico è il front-running, cioè quando un operatore vede una transazione in arrivo e ne inserisce una propria prima di essa, pagando commissioni più alte, per trarne vantaggio economico.

Il MEV non è solo un problema tecnico.

Ha conseguenze dirette sugli utenti, che possono subire perdite economiche o pagare commissioni più alte. Inoltre riduce la fairness del sistema, favorisce centralizzazione e può creare instabilità nella rete.

L'obiettivo del paper non è spiegare cos'è il MEV in generale, ma analizzare le principali strategie utilizzate per mitigarlo sia su Ethereum Layer 1 sia sulle Layer 2.

Il paper propone una classificazione delle strategie di mitigazione del MEV.

Le divide in due grandi categorie:

1. Prevention / Reduction

Queste tecniche cercano di eliminare o ridurre le opportunità di MEV prima che vengano sfruttate.

Per esempio:

- limitando la possibilità di riordinare le transazioni;
- impedendo ai bot di vedere le informazioni nel mempool;
- oppure rendendo economicamente sconsigliato il MEV.

2. Side-effect Reduction

Queste strategie non eliminano il MEV, ma cercano di ridurre gli effetti negativi.

L'idea è “democratizzare” il MEV, distribuendo le opportunità tra più partecipanti invece di lasciarle concentrate nelle mani di pochi validatori o bot.

L'obiettivo è quindi ridurre:

- congestione;
- instabilità della rete;

vantaggi eccessivi dei validatori.

Prevention / Reduction: Fair ordering policies

Uno dei principali approcci per mitigare il MEV riguarda le *fair ordering policies*.

L'idea di base è semplice: se il problema nasce dal fatto che i sequencer possono riordinare le transazioni a proprio vantaggio, allora bisogna imporre regole precise che limitino questa libertà.

Queste regole vengono integrate direttamente nel protocollo attraverso codice e smart contract, obbligando il sequencer a seguire criteri prestabiliti per decidere l'ordine delle transazioni.

Tuttavia, anche con queste politiche rimangono alcuni problemi:

- il sequencer potrebbe comunque censurare o ritardare transazioni;
- nei rollup con mempool private, il sequencer potrebbe divulgare informazioni ai bot esterni;
- nei sistemi con un solo sequencer serve comunque un forte livello di fiducia.

Single sequencer

Negli approcci a Single sequencer Un solo sequencer che ordina tutte le transazioni.

La politica più comune è la **FCFS**, cioè *First Come First Serve*.

- le transazioni vengono ordinate in base al momento di arrivo;
- chi arriva prima viene processato prima.

Però emergono nuovi problemi.

- **1. Latency wars:** Gli utenti cercano di ottenere connessioni sempre più veloci verso il sequencer per inviare le transazioni qualche millisecondo prima degli altri.
- **2. Spam attacks:** Un attaccante può inviare enormi quantità di transazioni per occupare la coda e rallentare gli utenti normali.

TimeBoost

Per migliorare il classico FCFS, il paper presenta **TimeBoost** che combina:

- il tempo di arrivo della transazione;
- e un sistema di offerte economiche.

Ogni transazione riceve un punteggio calcolato usando:

- timestamp;
- bid economico.

Chi offre di più può “comprare tempo” e migliorare la priorità della propria transazione.

La priorità di una transazione tx_i è:

$$S(t_i, b_i) = \pi(b_i) - t_i$$

dove:

- t_i = timestamp di arrivo della transazione
- b_i = bid (la “mancia” che l’utente paga per farsi avanti)
- $\pi(b_i)$ = funzione di priorità del bid

Gli utenti competono economicamente entro intervalli temporali limitati, TimeBoost limita la competizione solo a piccole finestre.

Decentralized Sequencing

Il paper analizza poi i sistemi con più sequencer, dove vari nodi collaborano per decidere l’ordine delle transazioni invece di affidarsi a un unico operatore.

Questo riduce centralizzazione e single point of failure, ma introduce problemi di sincronizzazione dovuti alla latenza di rete.

Protocolli come Themis e Wendy cercano di garantire un ordinamento fair usando leader temporanei oppure timestamp sincronizzati tra i nodi.

Tuttavia questi approcci richiedono molta coordinazione e aumentano la complessità e l’overhead della rete.

Prevention / Reduction: Privacy-preserving methods

La seconda grande area riguarda i **privacy-preserving methods**.

L’idea qui è diversa: si cerca di nascondere il contenuto delle transazioni finché l’ordine non è già stato deciso.

Se nessuno può leggere le transazioni nel mempool, diventa molto più difficile fare front-running.

Una delle tecniche principali è la threshold encryption.

Le transazioni vengono cifrate e la chiave di decifratura è distribuita tra più soggetti. Solo quando abbastanza partecipanti collaborano, la transazione può essere decifrata ed eseguita. In questo modo il validator non può vedere il contenuto della transazione prima di averne fissato l’ordine.

Un'altra tecnica importante è la delay encryption, considerata dal paper una delle direzioni più promettenti.

Qui la transazione resta cifrata per un certo periodo di tempo e la chiave può essere ottenuta solo dopo aver risolto un problema computazionale.

Quindi, anche se qualcuno vede la transazione, non può leggerla immediatamente.

Il paper discute anche i Trusted Execution Environments, o TEE, cioè aree hardware protette che eseguono codice in modo isolato.

Le transazioni vengono elaborate dentro enclave protette e pubblicate solo dopo l'esecuzione.

La sicurezza dipende quindi dall'hardware stesso.

Il vantaggio è che:

- la gestione delle chiavi è automatica;
- i dati restano nascosti anche al sistema operativo.

Lo svantaggio è che bisogna fidarsi del produttore hardware e della sicurezza fisica del dispositivo.

Prevention / Reduction: Smart contract

Successivamente il paper analizza le protezioni a livello di **smart contract**.

L'obiettivo è ridurre la possibilità di front-running direttamente dentro le applicazioni.

Le soluzioni si dividono in due grandi famiglie.

- La prima è completamente on-chain e resta dentro gli smart contract.
- La seconda famiglia invece introduce componenti off-chain, cioè fuori dalla blockchain, ma comunque legati al sistema. Un esempio importante è CoW Swap.

La protezione a livello di smart contract non elimina il MEV alla radice, ma lo "circonda": limita le opportunità di attacco direttamente nel comportamento delle applicazioni, usando batch, intenti, aste e meccanismi di mercato più controllati.

Side-effect reduction: Proposer-Builder Separation

Infine viene discusso il **Proposer-Builder Separation**, o PBS.

Separare:

- chi **propone un blocco: proposer**, cioè i validator, che non costruiscono il blocco ma si limitano a scegliere tra quelli già pronti.
- da chi **lo costruisce: block builder**, che costruiscono interi blocchi cercando di massimizzare il profitto includendo transazioni e MEV

L'implementazione più importante oggi è **MEV-Boost**, una soluzione off-chain che già viene utilizzata per circa il 90% dei blocchi su Ethereum Proof of Stake.

In questo modello:

- i searcher trovano opportunità di MEV;
- i builder costruiscono i blocchi;
- i relay verificano i blocchi;
- e i validator scelgono quale proporre.

Il proposer firma soltanto l'header del blocco senza vedere il contenuto completo, riducendo così la possibilità di manipolare direttamente l'ordine delle transazioni.

Questo approccio riduce il potere del singolo validator, ma introduce nuovi rischi di centralizzazione nei relay e nei builder.

Per concludere, il messaggio principale del paper è che non esiste una soluzione unica al problema del MEV.

Ogni strategia risolve una parte del problema ma introduce nuovi compromessi tra:

- decentralizzazione
- privacy
- latenza
- complessità
- scalabilità

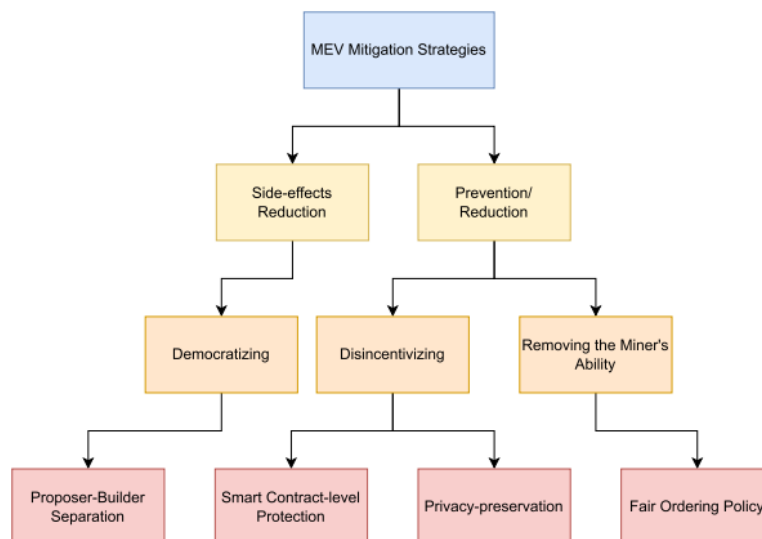
Il fair ordering cerca di controllare l'ordine delle transazioni.

Le soluzioni privacy-preserving cercano invece di nascondere il contenuto delle transazioni.

PBS separa i ruoli nella costruzione dei blocchi, mentre lo shared sequencing prova a ridurre la frammentazione tra chain diverse.

Il messaggio finale del paper è che il MEV non può essere eliminato completamente, perché nasce direttamente dalla struttura delle blockchain basate sull'ordinamento delle transazioni.

Le soluzioni attuali cercano quindi di limitarlo, redistribuirlo oppure renderne l'estrazione più difficile e costosa.



Seminar on the Paper “MEV Mitigation Approaches in Ethereum and Layer-2 Chains”

Good morning everyone.

In this seminar, I will discuss the paper “*MEV Mitigation Approaches in Ethereum and Layer-2 Chains: A Comprehensive Survey*”, published in 2024 in IEEE Access. Because apparently humanity decided distributed finance was not complicated enough without turning transaction ordering into a competitive sport.

MEV occurs when miners or validators manipulate the order of transactions within a block in order to obtain personal profit. A typical example is front-running, where an operator detects an incoming transaction and inserts their own transaction before it by paying higher fees, in order to gain an economic advantage.

MEV is not only a technical issue.

It has direct consequences for users, who may suffer financial losses or pay higher transaction fees. In addition, it reduces the fairness of the system, encourages centralization, and can create instability in the network.

The objective of the paper is not to explain what MEV is in general, but rather to analyze the main strategies used to mitigate it both on Ethereum Layer 1 and on Layer 2 chains.

The paper proposes a classification of MEV mitigation strategies.

These strategies are divided into two main categories:

1. **Prevention / Reduction**

These techniques aim to eliminate or reduce MEV opportunities before they can be exploited.

For example:

- limiting the possibility of reordering transactions;
- preventing bots from accessing information in the mempool;
- or making MEV extraction economically unprofitable.

2. **Side-effect Reduction**

These strategies do not eliminate MEV, but instead try to reduce its negative effects.

The idea is to “democratize” MEV by distributing opportunities among more participants instead of concentrating them in the hands of a few validators or bots.

The goal is therefore to reduce:

- congestion;
- network instability;
- excessive validator advantages

Prevention / Reduction: Fair Ordering Policies

One of the main approaches to mitigating MEV involves fair ordering policies.

The basic idea is simple: if the problem arises because sequencers can reorder transactions for their own benefit, then precise rules must be introduced to limit this freedom.

These rules are integrated directly into the protocol through code and smart contracts, forcing the sequencer to follow predefined criteria when deciding transaction order.

However, even with these policies, several problems remain:

- the sequencer could still censor or delay transactions;
- in rollups with private mempools, the sequencer could leak information to external bots;
- in systems with a single sequencer, a strong level of trust is still required.

Single Sequencer

In single sequencer approaches, one sequencer is responsible for ordering all transactions.

The most common policy is FCFS, meaning *First Come First Serve*:

- transactions are ordered according to their arrival time;
- whoever arrives first is processed first.

However, new problems emerge:

1. Latency wars

Users try to obtain increasingly faster connections to the sequencer in order to submit transactions a few milliseconds earlier than competitors. Humanity reinventing Formula 1, except the cars are servers and everyone is somehow more annoying.

2. Spam attacks

An attacker can submit enormous numbers of transactions to fill the queue and slow down normal users.

TimeBoost

To improve the traditional FCFS mechanism, the paper presents **TimeBoost**, which combines:

- transaction arrival time;
- and an economic bidding system.

Each transaction receives a score calculated using:

- a timestamp;
- and an economic bid.

Users willing to pay more can effectively “buy time” and improve the priority of their transaction.

The priority of a transaction tx_{itx_itxi} is:

$$S(t_i, b_i) = \pi(b_i) - t_i$$

dove:

- t_i = transaction arrival timestamp
- b_i = bid, meaning the tip paid by the user to gain priority
- $\pi(b_i)$ = bid priority function

Users compete economically within limited time intervals, so TimeBoost restricts competition to small windows instead of allowing continuous latency wars.

Decentralized Sequencing

The paper also analyzes systems with multiple sequencers, where several nodes collaborate to decide transaction ordering instead of relying on a single operator.

This reduces centralization and removes the single point of failure, but introduces synchronization problems caused by network latency.

Protocols such as Themis and Wendy attempt to guarantee fair ordering by using temporary leaders or synchronized timestamps across nodes.

However, these approaches require significant coordination and increase both complexity and network overhead.

Prevention / Reduction: Privacy-Preserving Methods

The second major area concerns privacy-preserving methods.

The idea here is different: transaction contents are hidden until the ordering has already been decided. If nobody can read transactions in the mempool, front-running becomes much more difficult.

One of the main techniques is **threshold encryption**.

Transactions are encrypted, and the decryption key is distributed among multiple parties. Only when enough participants cooperate can the transaction be decrypted and executed. In this way, the validator cannot view the transaction contents before fixing its order.

Another important technique is **delay encryption**, considered by the paper to be one of the most promising directions.

Here, the transaction remains encrypted for a certain period of time, and the key can only be obtained after solving a computational problem. Therefore, even if someone sees the transaction, they cannot immediately read it.

The paper also discusses **Trusted Execution Environments**, or TEEs, which are protected hardware areas that execute code in isolation.

Transactions are processed inside protected enclaves and published only after execution. Security therefore depends on the hardware itself.

The advantages are:

- automatic key management;
- data remains hidden even from the operating system.

The disadvantage is that users must trust the hardware manufacturer and the physical security of the device. Which is always comforting. “Trust us, the secret computation box is definitely secure.”

Civilization built entire financial systems on statements like that.

Prevention / Reduction: Smart Contract Solutions

The paper then analyzes protections implemented at the smart contract level.

The objective is to reduce the possibility of front-running directly inside applications.

These solutions can be divided into two main families:

- the first is entirely on-chain and remains inside smart contracts;
- the second introduces off-chain components connected to the blockchain ecosystem. An important example is CoW Swap.

Smart contract protections do not eliminate MEV at its root, but instead “surround” it by limiting attack opportunities directly in application behavior through batches, intents, auctions, and more controlled market mechanisms.

Side-effect Reduction: Proposer-Builder Separation

Finally, the paper discusses **Proposer-Builder Separation**, or PBS.

The idea is to separate:

- the entity proposing the block, namely the proposer or validator, who only selects among already prepared blocks;
- from the entity building the block, namely the block builder, who constructs blocks while trying to maximize profit through transaction inclusion and MEV extraction.

The most important implementation today is MEV-Boost, an off-chain solution currently used for roughly 90% of blocks on Ethereum Proof of Stake.

In this model:

- searchers identify MEV opportunities;
- builders construct blocks;
- relays verify blocks;
- validators choose which block to propose.

The proposer signs only the block header without seeing the full block contents, reducing the possibility of directly manipulating transaction ordering.

This approach reduces the power of individual validators, but introduces new centralization risks around relays and builders. Humans solving centralization by inventing additional centralized intermediaries. An eternal classic.

Conclusion

To conclude, the main message of the paper is that there is no single solution to the MEV problem.

Every strategy solves part of the issue while introducing new trade-offs involving:

- decentralization;
- privacy;
- latency;
- complexity;
- scalability.

Fair ordering attempts to control transaction ordering.

Privacy-preserving solutions instead aim to hide transaction contents.

PBS separates roles in block construction, while shared sequencing attempts to reduce fragmentation across different chains.

The paper's final conclusion is that MEV cannot be completely eliminated, because it emerges directly from the structure of blockchains based on transaction ordering itself.

Current solutions therefore aim to limit MEV, redistribute it, or make its extraction more difficult and expensive. Which, in computer science terms, is often the closest thing humanity gets to peace.